

Project #3 CS 3410– Spring 2026 <Emmett Bicknell & Brandon Aikman>

Requirements

Restate the problem specification, and any detailed requirements

We were tasked with solving the Galactic Breakup problem using a disjoint set data structure. The Galactic Breakup problem describes a 3D 'empire' which breaks up into separate monarchies with a monarchy separating each month, and we need to determine the amount of months during the breakup that the empire is disconnected. The problem is given by a user inputting the size of the empire (n by m by k) and how many secessions will occur (l). Then, l lines follow which describe the count of dominions (p) and the dominions (d_i) leaving the empire for each month, in order. Because we utilized a disjoint set data structure, we also had to implement three methods: `findSet`, `unionSets`, `makeSet`, as referenced in Design.

Design

How did you attack the problem? What choices did you make in your design, and why? Show class diagrams for more complex designs.

To begin, we made a `Node` class. This class represents the disjoint sets, though because we used a node structure, we found that `Node` was a helpful name. This node implements each of the required disjoint set methods. We made sure to use path compression in the `findSet` method, as well as union by rank. These ensured optimal runtime. Once we had these methods created, we created the `Execute` class which reads in the input, puts it in the right order, and then performs the disjoint set operations.

Security Analysis

State the potential security vulnerabilities of your design. How could these vulnerabilities be exploited by an adversary? What would be the impact if the vulnerability is exploited?

There are no known security vulnerabilities. The program cannot run any powershell commands so nothing bad can happen.

Implementation

Outline any interesting implementation details.

Instead of using an $n \cdot m \cdot k$ array of arrays of arrays, we decided to use one-dimensional array of size $n \cdot m \cdot k$. Then, instead of having to calculate the coordinates of each node, we could simply store dominion a at `array[a]`. This then made the `findAdjacencies` method slightly more complicated. Instead of changing one coordinate by $1, m, k$ we instead could increment or decrement the index by $1, n, m$ or $n \cdot m$ to move in the n, m or k direction, respectively.

Because of this, we created a `findAdjacencies` method that calculated the indices of the nodes adjacent to a given node. Then, the `checkAdjacencies` method checks each of these indices to see if it holds a valid node,

and if so, unions the sets if needed.

We used a Stack of Node arrays to easily reverse the order of inputs.

Testing

Explain how you tested your program, enumerating the tests if possible. Explain why your test set was sufficient to believe that the software is working properly, i.e., what were the range of possibilities of errors that you were testing for.

We used the test cases found in Gradel. These covered a range of problems and hence seemed to provide sufficient testing. When the test cases failed, if an exception was thrown, we used the error message to identify where issues were occurring so that they could be easily fixed. Once no exceptions were being thrown, debugging was more difficult. To catch logic issues, we then set breakpoints and walked through the execution, line-by-line, until we found unexpected behaviour. We were then able to fully fix the program.

AI Use

How did you use generative AI in this project? Be specific!

We did not use generative AI on this project.

Summary/Conclusion

Present your results. Did it work properly? Are there any limitations? If it is an analysis-type project, this section may be significantly longer than for a simple implementation-type project.

The program worked as expected and passed all test cases. As far as we know it has no limitations. We specifically designed it to be able to correctly address cases where not all dominions are removed. We do assume that the input is correctly formatted.

Lessons Learned

List any lessons learned, especially in regards to AI use.

This project was a reminder of how helpful the debugger is when trying to find logic errors. At one point, the program was not throwing any exceptions but was giving an incorrect output. In order to figure out where issues were coming from, we had to go line-by-line through the execution of the program. Doing this helped us to understand what was happening in the program.

Time Spent

Approximately how much time did you spend on this project?

We spent about 12 hours on this project.